

动手学大模型应用开发 · 第 2 节

Agent 组成 与最小工具循环

从系统组成到决策策略与可靠运行

主讲：陈思州



理论主线 · 从结构到策略

先建立共同定义，再理解系统循环、决策策略与可靠运行。

01 定义与边界

Agent 与 Workflow 的差异，以及何时值得使用

02 组成与循环

模型、运行程序、工具、状态如何形成闭环

03 决策策略

ReAct、规划与反思的作用边界

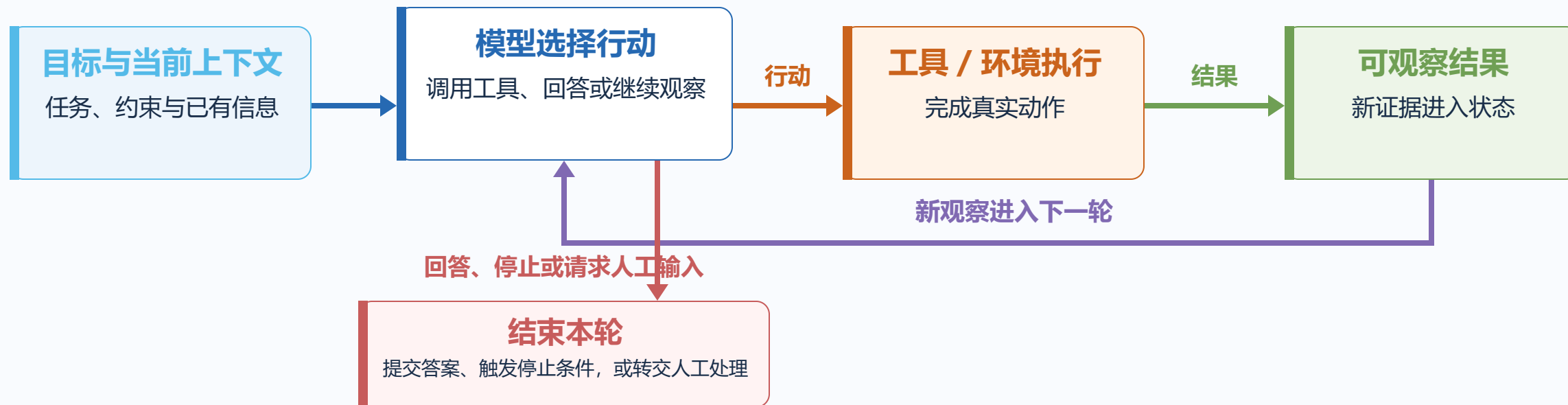
04 可靠运行

上下文、权限、人工控制与可观测性

先建立最小循环，再理解规划、反思与可靠运行如何扩展这套结构。

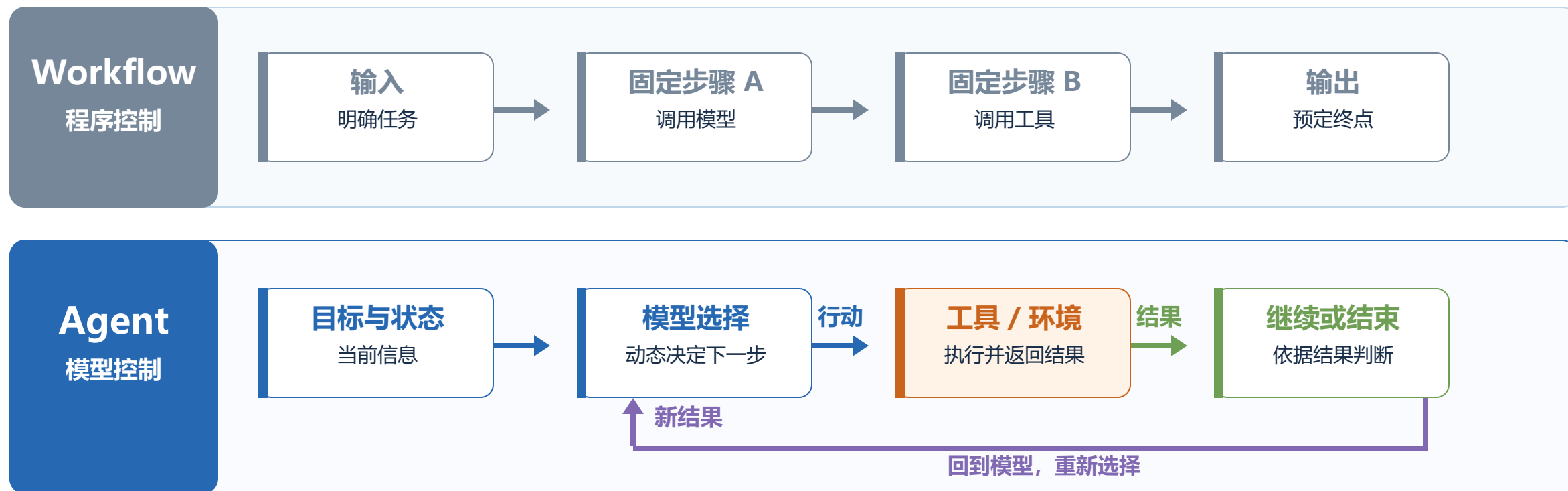
Agent 让模型在循环中选择下一步行动，并根据环境结果继续、结束或请求人工输入。

最小行动闭环



判断是否为 Agent，关键看模型是否根据环境结果动态决定下一步。

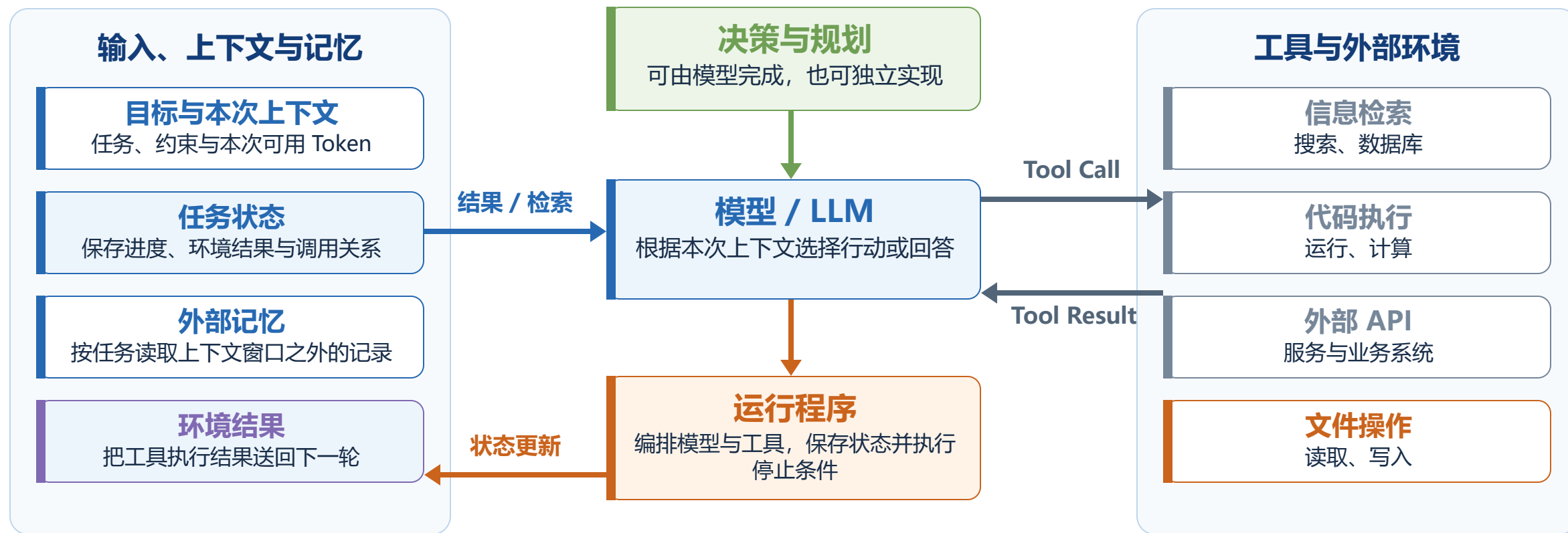
两者都可以调用模型和工具，差别在于下一步由预定义代码还是模型动态决定。



Workflow 适合路径明确、追求可预测性的任务；Agent 适合步骤无法预先写死、需要根据新信息调整的任务。

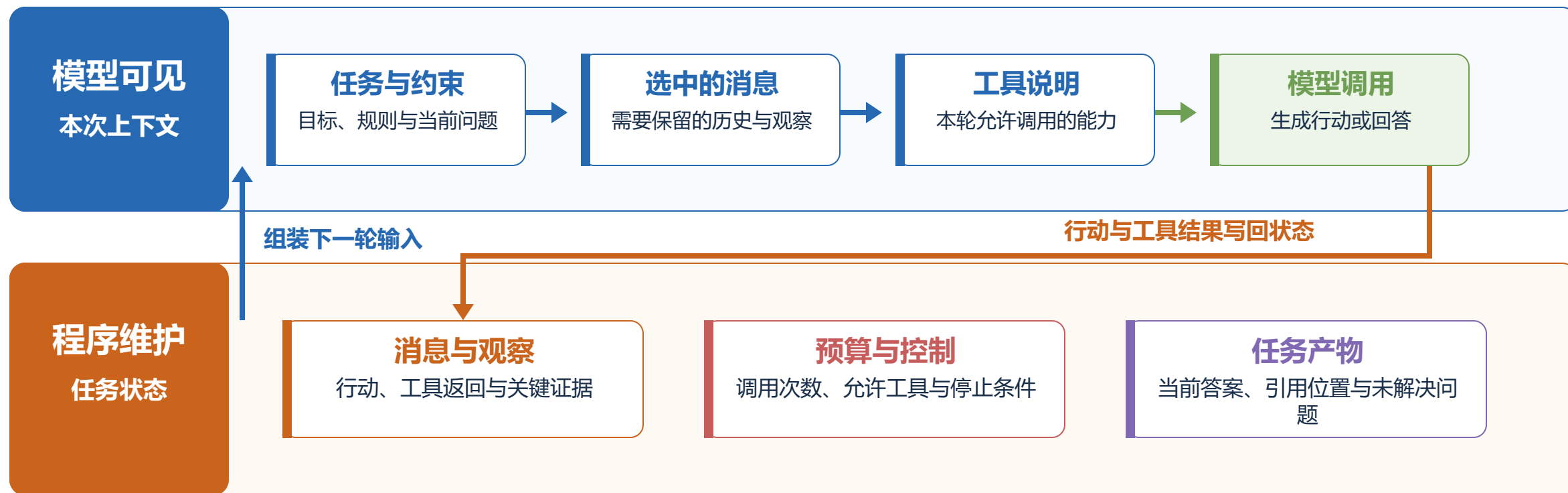
一个 Agent 系统由什么组成

一个可运行的 Agent 系统通常包含模型、运行程序、上下文与状态，以及工具和外部环境。



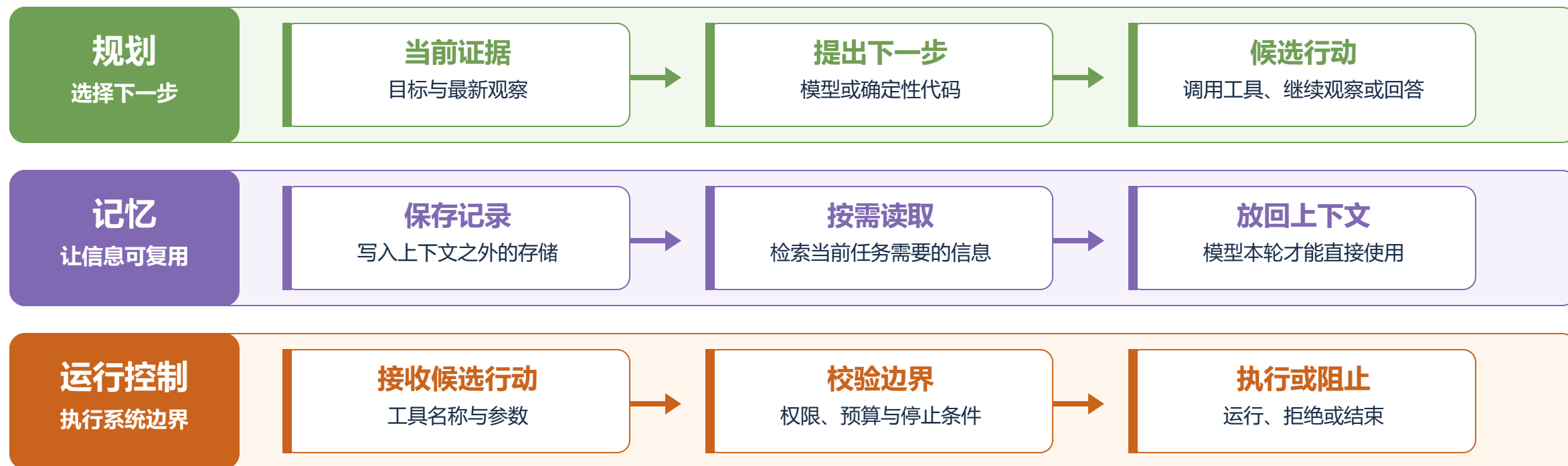
规划可以由模型在循环中完成，也可以作为独立模块。并非每个 Agent 都需要单独部署规划器。

模型只读取本次请求中的上下文；运行程序维护任务状态，并决定哪些信息进入下一轮。



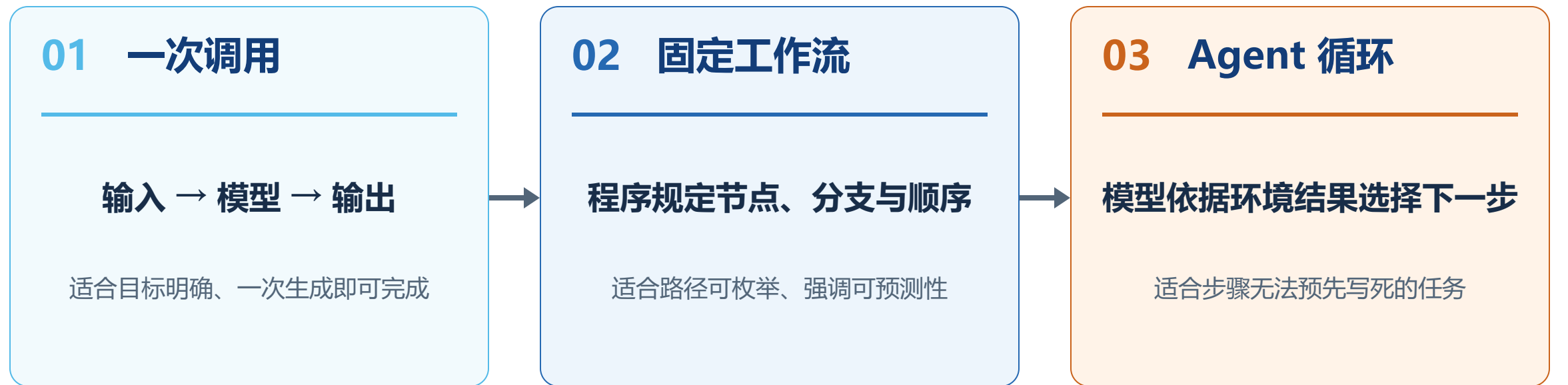
运行状态不会自动进入模型；运行程序必须选择并重新放入下一轮上下文。

规划选择下一步，记忆让信息跨轮次可用，运行控制决定动作能否真正执行。



计划提供候选行动，记忆提供可重新读取的信息，运行控制决定行动是否执行。

系统复杂度应当随任务需要逐级增加，而不是默认从 Agent 开始。



低成本 · 高可预测

高灵活性 · 需要更多控制

能用一次调用解决就不引入循环；路径可枚举时优先 workflow；只有步骤无法预先写死时才需要 Agent。

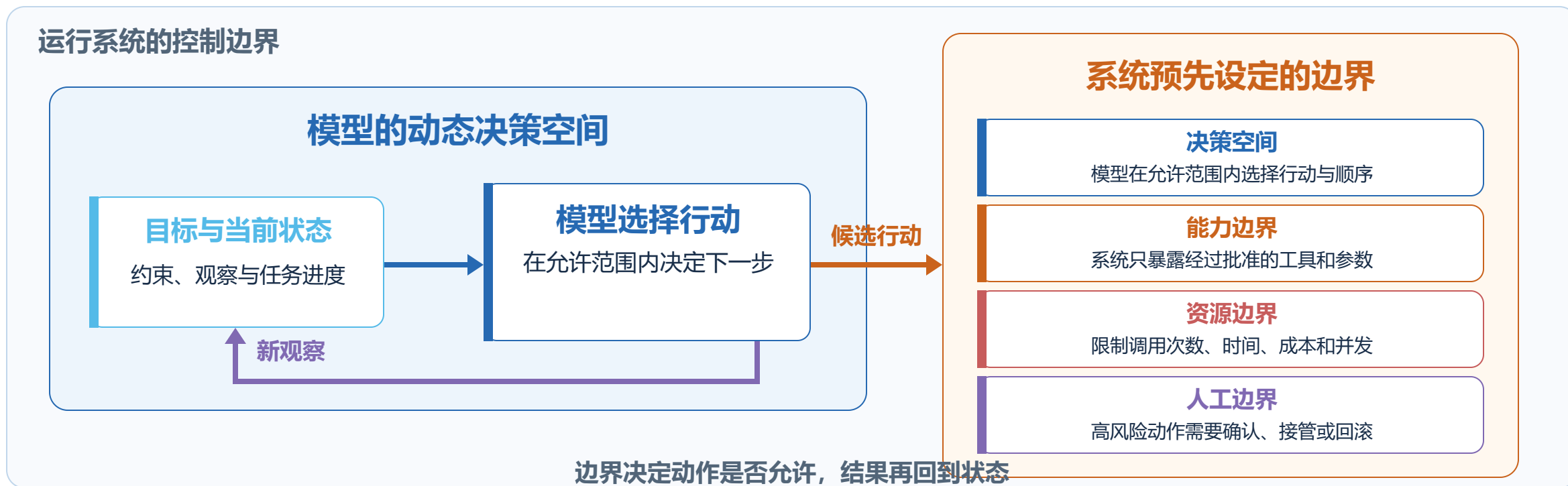
什么时候才需要 Agent

先判断路径与观察，再检查结果能否验收、环境是否受控。



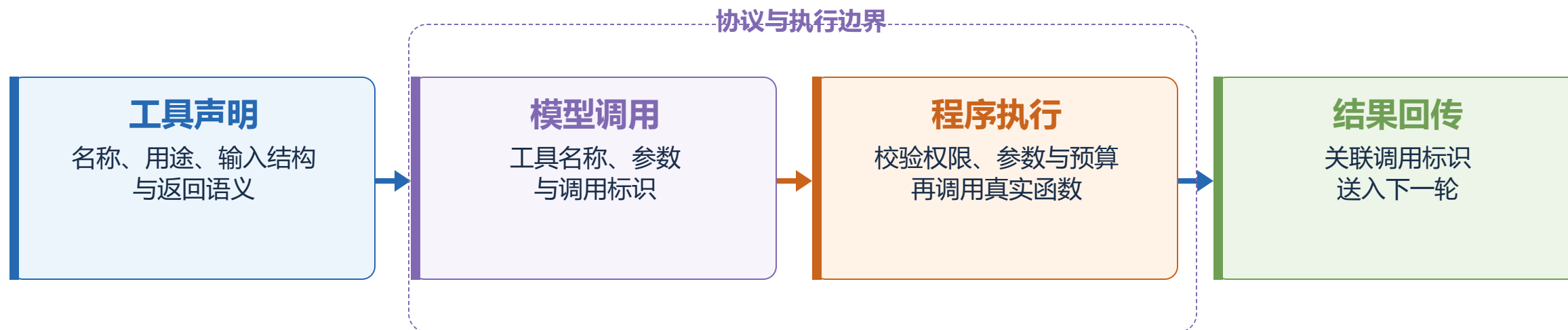
需要动态决策只是起点；只有同时具备可验收结果和受控环境，Agent 才是合理选择。

Agent 可以动态选择下一步，但运行系统必须约束权限、预算、工具范围和人工介入点。



工程上的 Agent 是受控自主系统。模型决定如何推进，运行系统决定哪些动作可以发生。

Function Calling 让模型用结构化调用表达意图；真实函数仍由运行程序执行。



Function Calling 是模型与运行程序之间的结构化协议，不是把代码执行权交给模型。

调用建议不等于已经执行

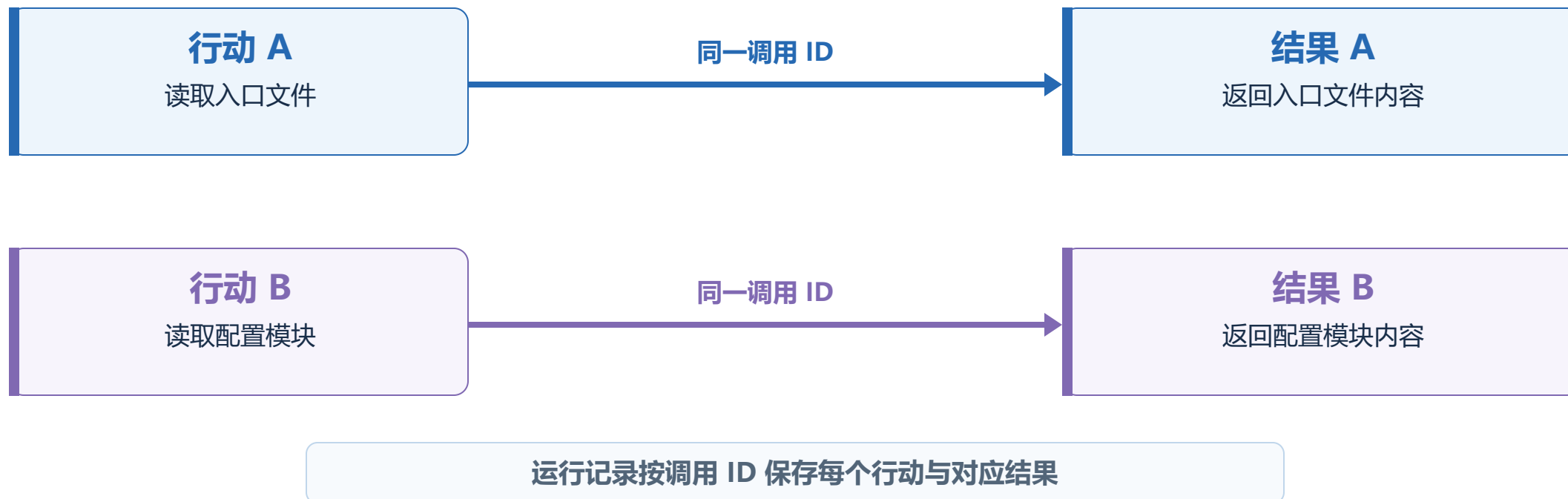
模型负责提出行动，运行程序负责判断行动是否允许并真正执行。



模型只能提出调用建议；运行程序负责权限校验、函数分派和异常处理。

行动与结果按调用 ID 对应

同一种工具可以连续执行多次，调用 ID 让后续决策知道每个结果来自哪一次行动。



每次行动与结果必须通过调用 ID 对应，否则下一轮可能使用错误的观察。

工具结果必须区分成功、空结果与失败

工具返回值既是环境证据，也是下一步决策的输入。模糊的错误会让模型误判任务状态。

成功

获得真实数据
同时保留来源或位置

空结果

调用有效，但没有命中
可以换线索继续取证

调用失败

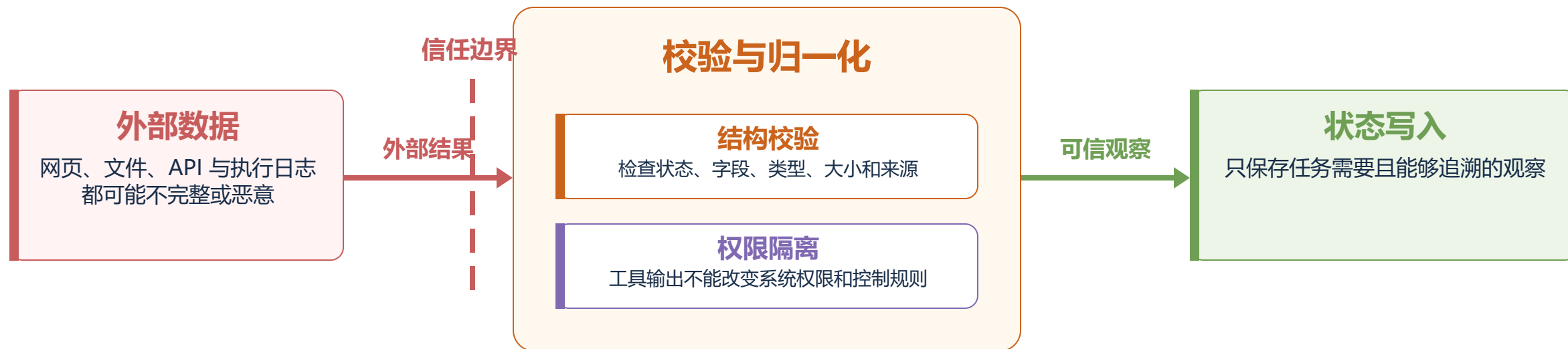
输入、权限或运行异常
先修正调用再继续

共同安全边界

工具输出始终按外部数据处理；其中的文字不能覆盖系统指令、权限或停止条件。

没有命中不等于调用失败；只有把结果类型讲清楚，循环才能选择正确的下一步。

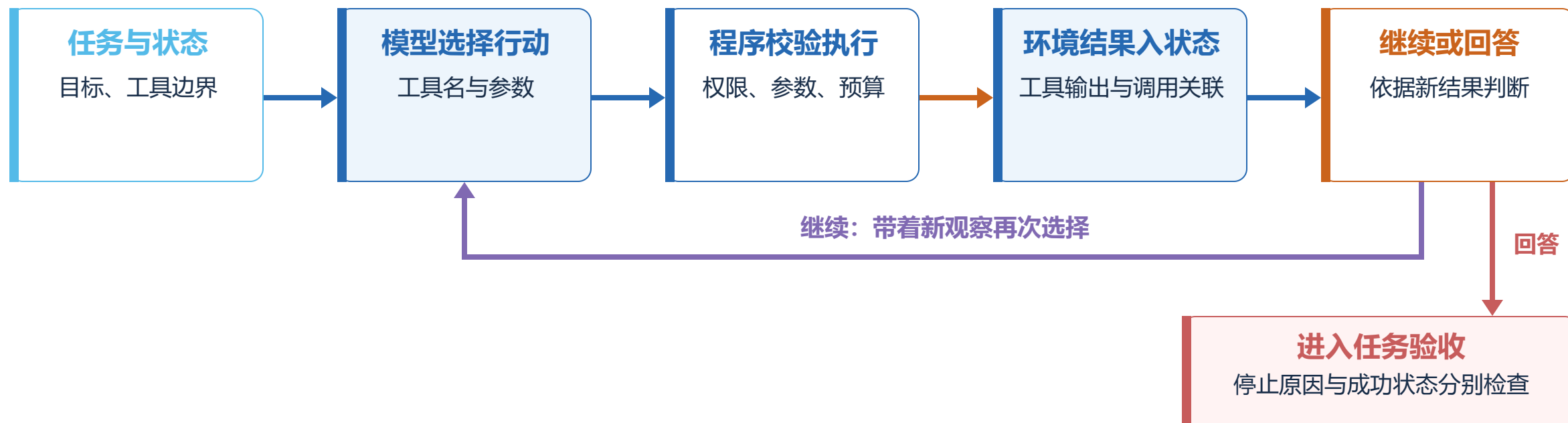
工具结果来自外部环境。运行程序需要验证来源、结构与权限，不能把返回文本直接当作更高优先级的指令。



工具输出提供数据，不能覆盖系统指令、权限、预算和停止条件

Agent 可以利用工具结果，但运行程序必须把结果当作不可信数据处理。

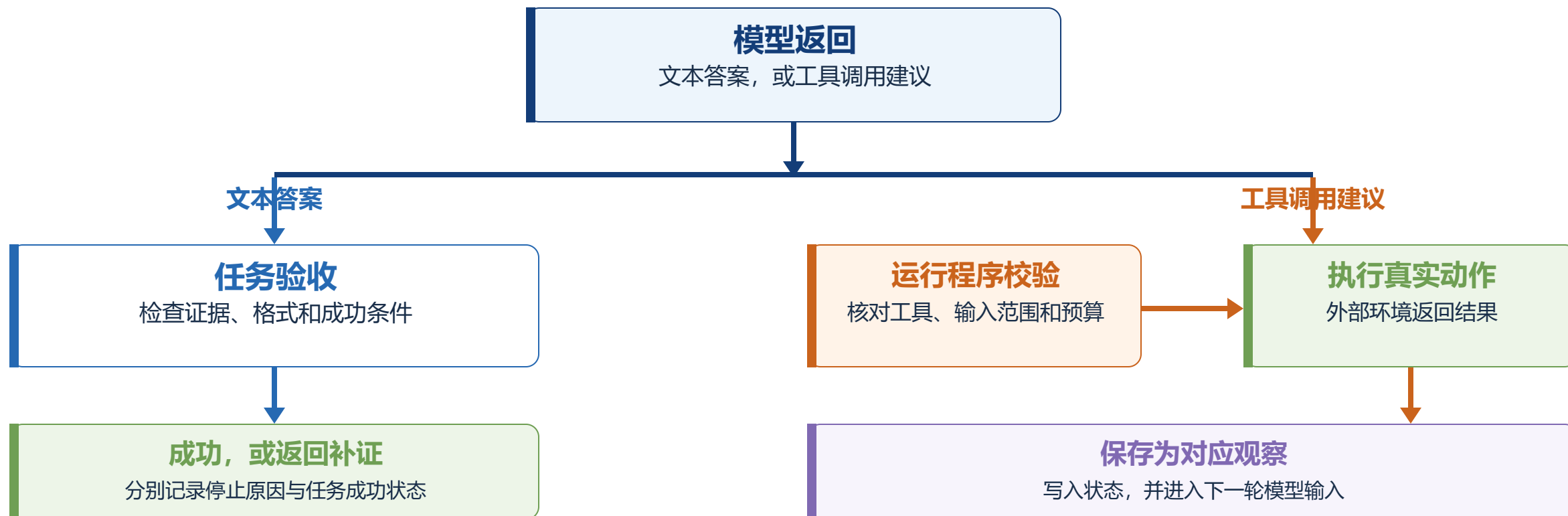
Agent 循环让模型选择行动，环境返回结果，结果再影响下一次选择。



课程检查工具调用、环境结果、状态和最终产物，不要求暴露模型的私有推理。

模型返回后的两条处理路径

模型返回文本时进入任务验收，返回工具调用建议时由运行程序校验并执行。



文本答案进入任务验收；工具结果写入状态，并成为下一轮模型输入。

停止原因	是否证明任务成功
模型给出最终回答	还要检查任务要求与依据
达到轮数 / 工具预算	不能证明
连接异常 / 协议错误	不能证明

例：模型没读文件就给出最终回答，运行可以结束，但“找到真实入口”仍未验收。

把 stop_reason 与 task_success 分开记录，才能区分提前作答、资源耗尽和真正完成。

最小循环能否可靠运行，取决于行动、环境结果和下一轮输入是否连得起来。

01

行动可执行

运行程序能够识别并检查候选能力

02

结果可追溯

环境结果能够对应到产生它的行动

03

观察可继续使用

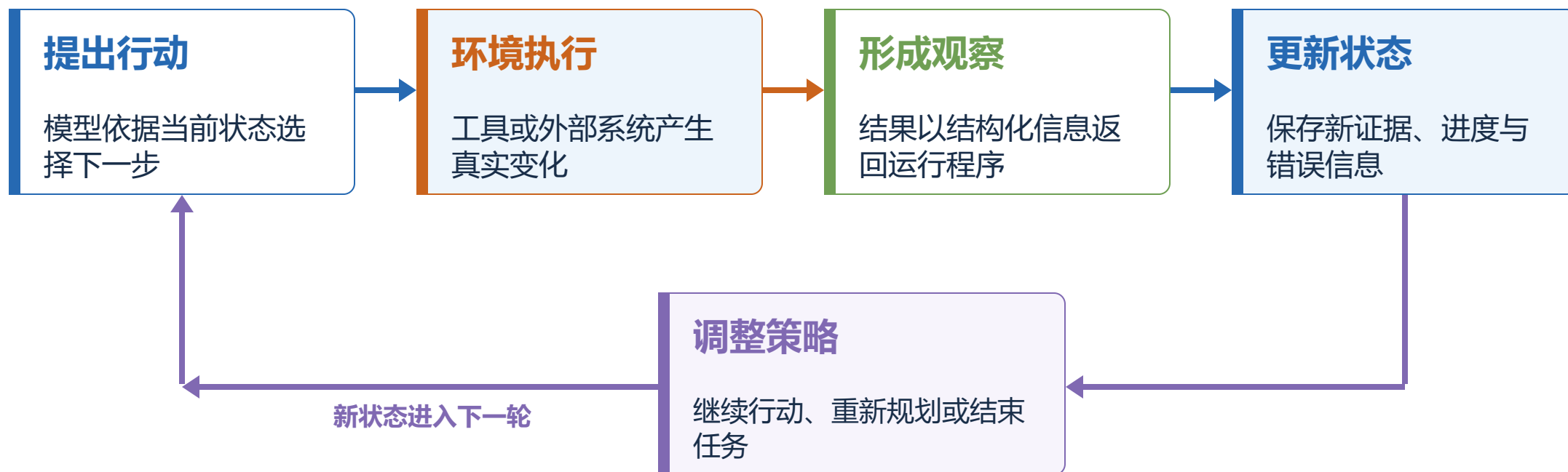
下一轮能够读取刚获得的新信息

三项都成立，循环才保持连续、可检查

行动、结果与下一轮输入连成闭环后，环境信息才会真正改变后续决策。

反馈闭环

环境结果写入状态后，下一轮决策才会随之变化。



Agent 的适应性来自环境反馈进入状态，并真正改变下一次决策。

ReAct 交替组织面向任务的推理与行动：行动取得环境观察，观察再更新后续计划与回答。



课堂可检查轨迹

工具调用 · 工具结果 · 状态变化 · 最终产物

可核验的轨迹包括决策摘要、工具调用、环境结果和最终产物。

优势

环境观察可以持续修正行动。工具让模型获得外部信息，并把推理与真实结果连接起来。

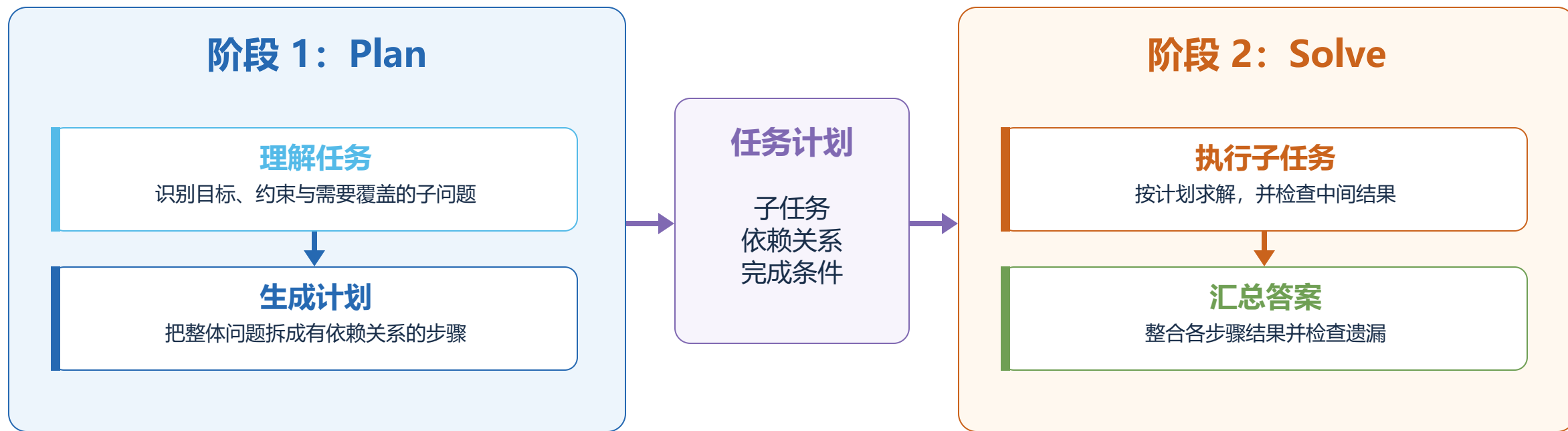
局限

长循环会累积成本和错误。模型还可能重复行动、误读观察，或在证据不足时提前回答。

第一次调用失败提供了“path 类型不正确”的观察，后续可以修正参数或改用搜索。

这是教学回放中的轨迹，工具实际执行。它证明循环能接收错误结果，不证明真实模型一定会这样修正。

Plan-and-Solve 先把复杂问题拆成子任务，再按计划逐项求解，重点减少遗漏步骤。



环境反馈改变前提时，需要重新规划，而不是机械执行原计划

Plan-and-Solve 强调先形成全局结构。若环境反馈改变前提，系统还需要重新规划能力。

Planner 负责维护任务结构，Executor 负责完成当前步骤。观察结果可以触发计划更新。



稳定计划减少局部漂移，重新规划让系统适应环境变化。

Reflection 的核心是利用反馈改进下一轮。反馈可以来自同一模型、独立 Critic Agent 或外部验证。

单模型自我反思

同一模型先生成结果，再切换到反馈提示并完成修订

生成结果
初稿或行动结果

同一模型生成反馈
切换到反馈提示

根据反馈修订
仍由同一模型完成

独立 Critic Agent

执行 Agent 负责生成，Critic Agent 按标准独立评估

执行 Agent 输出
负责生成与行动

Critic Agent 评估
独立检查标准与证据

执行 Agent 修订
反馈进入下一轮

外部验证反馈

测试、规则、环境结果或人工意见提供可核验的反馈

执行 Agent 行动
产生结果与轨迹

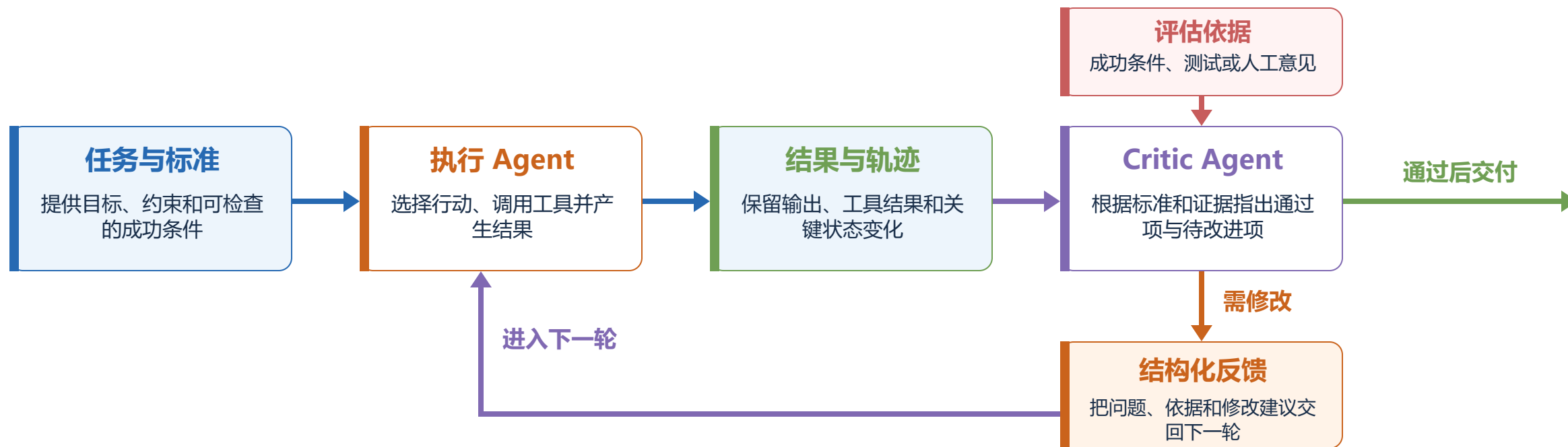
外部验证
测试、规则、环境或人工

根据证据修订
避免只凭自我判断

是否增加第二个 Agent，取决于评估独立性、任务风险和调用成本。

第二个 Agent 可以提高评估独立性，但任何反思都需要明确标准和真实证据。

执行 Agent 负责行动，Critic Agent 依据验收标准和运行证据评估结果，再把反馈交回下一轮。



若把语言反馈写入情景记忆并在后续任务中检索，就形成 Reflexion 一类跨尝试机制。

Critic 分离了生成与评估职责，但 Critic 本身仍需要标准、证据和校验。

ReAct、Plan-and-Solve 与 Reflection 解决的问题不同，选择策略要看任务结构与反馈时机。

行动过程中

ReAct

外部观察频繁改变下一步，适合边行动边更新

行动开始前

Plan-and-Solve

子任务结构较清楚，适合先建立全局计划

一次尝试之后

Reflection

一次尝试结束后有明确反馈，适合总结并改进

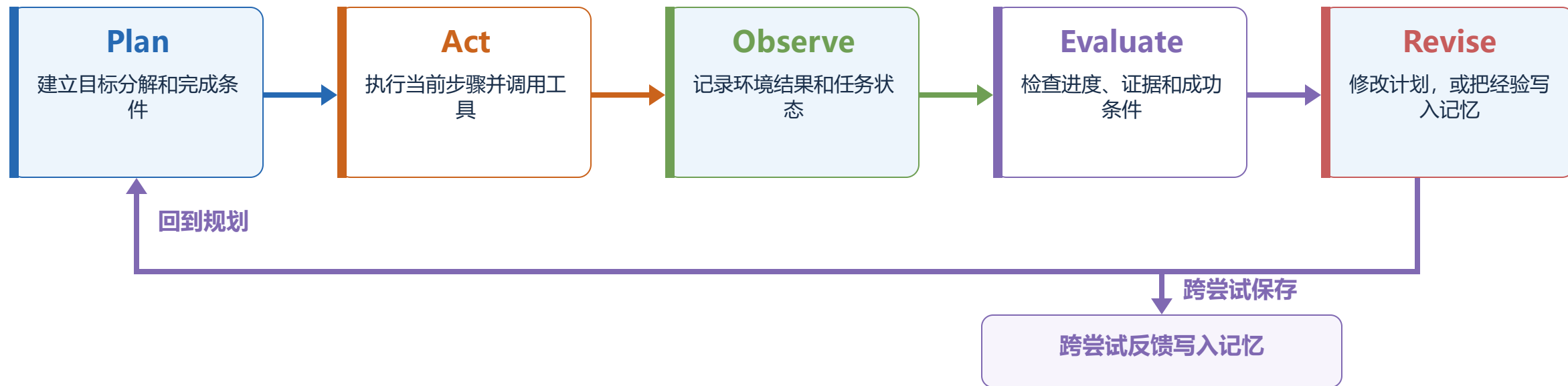
组合使用

复杂任务常同时需要规划、行动反馈和复盘

策略选择取决于反馈出现的时机和任务结构，复杂任务也可以组合多种策略。

规划、行动与反思的组合循环

复杂 Agent 可以先规划，再通过工具行动获取观察，并在检查失败后修正计划或保存经验。



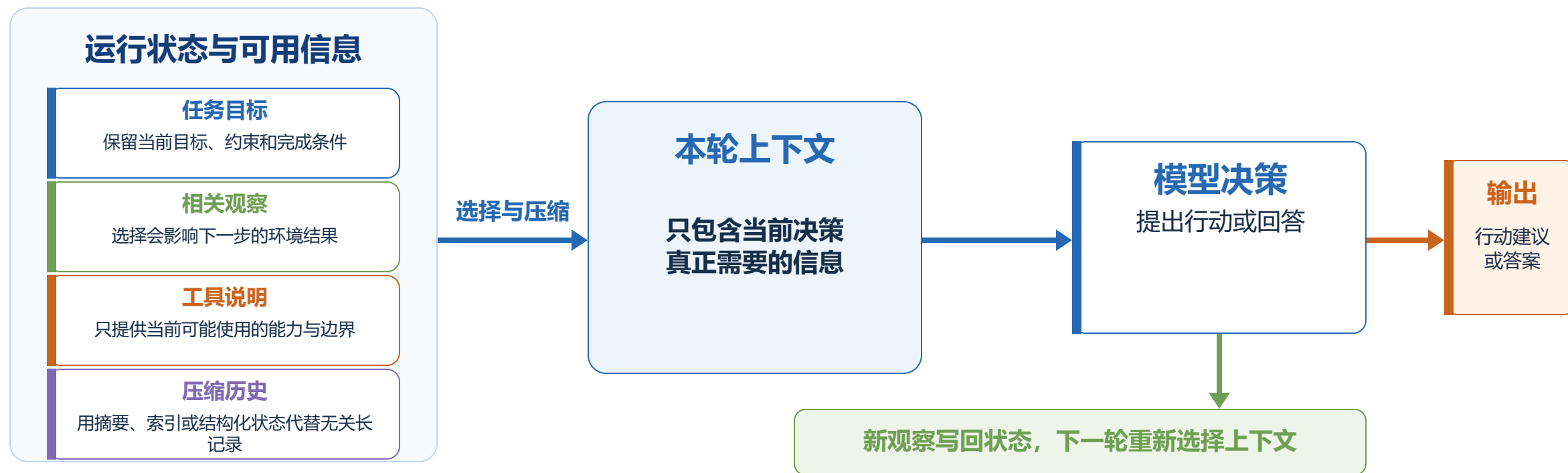
组合策略的核心不是增加更多模型调用，而是让每次调用承担明确职责。

比较维度	ReAct	Plan-and-Solve	Reflection
主要时机	行动过程中	行动开始前	一次尝试之后
核心输入	最新环境观察	任务结构与约束	结果反馈与失败信息
主要作用	根据观察调整行动	组织步骤并减少遗漏	改进后续尝试
主要风险	循环漂移与成本累积	初始计划错误	错误反思或记忆污染

同是“没有得到内容”，可能来自非法调用、拒绝读取，也可能是合法搜索的空结果。

恢复策略不同：修正参数、回到授权范围或换检索词。失败分类应保存在可回查记录里。

Agent 的状态可能很大，但模型每一轮只应看到当前决策需要的信息。



更长的上下文不一定更好。关键是让模型看到足以做出当前决策的信息。

动作风险越高，系统需要越强的人类控制。高影响、不可逆或难以验收的动作尤其如此。

01 只读观察

检索、读取和分析通常可以自动执行

02 可逆修改

保存草稿或创建临时产物，需要审计与回滚

03 外部影响

发送消息、发布内容或修改共享系统需要确认

04 高风险动作

付款、删除、权限变更应限制或强制人工接管

低风险 • 高可逆

高影响 • 需要确认或接管

合理的自主性来自分级授权，而不是一次性把所有能力交给模型。

仅有最终答案不足以解释执行过程。运行轨迹还应记录模型请求、工具调用、状态变化和停止原因。



一次任务的时间线

模型记录	输入摘要、输出类型、模型版本与耗时
工具记录	能力名称、参数、调用 ID、结果状态与来源
状态记录	关键观察、计划进度、预算和产物变化
结束记录	停止原因、任务成功状态和人工介入

没有结构化轨迹，只能看到 Agent 失败了，却很难知道问题发生在哪里。

设计 Agent 时，先明确任务、边界与验收，再决定模型怎样在循环中选择行动。



先明确任务、工具、状态、控制和验收，再选择 ReAct、规划或反思等策略。

这一节，我们学会了什么

知识结构 · Agent 理论主线

定义、组成、循环、策略与可靠运行形成一套完整框架

✓ 定义

模型依据环境反馈动态选择下一步行动

✓ 结构

运行程序连接模型、工具、状态与安全边界

✓ 策略

ReAct、规划与反思解决不同类型的问题

✓ 可靠性

上下文、权限、人工控制和轨迹共同约束运行

先建立受控的最小循环，再按任务需要增加规划、反思与长期运行能力。

本节的定义、策略与工程边界主要参考以下论文和官方资料。

- | | | |
|-----|----------------------|---|
| [1] | Yao, Shunyu, et al. | ReAct: Synergizing Reasoning and Acting in Language Models
arXiv:2210.03629 |
| [2] | Wang, Lei, et al. | Plan-and-Solve Prompting
ACL 2023 · 10.18653/v1/2023.acl-long.147 |
| [3] | Madaan, Aman, et al. | Self-Refine: Iterative Refinement with Self-Feedback
NeurIPS 2023 |
| [4] | Shinn, Noah, et al. | Reflexion: Language Agents with Verbal Reinforcement Learning
NeurIPS 2023 |
| [5] | Du, Yilun, et al. | Improving Factuality and Reasoning through Multiagent Debate
ICML 2024 |
| [6] | Anthropic | Building Effective AI Agents
anthropic.com/engineering/building-effective-agents |
| [7] | Anthropic | Effective Context Engineering for AI Agents
anthropic.com/engineering/effective-context-engineering-for-ai-agents |
| [8] | OpenAI | Function calling
developers.openai.com/api/docs/guides/function-calling |